Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота №5

«Нормалізація бази даних»

Роботу виконали
Студенти групи ІО-45, ІО-43
Ренцевич Д. , Гапон А.
Роботу перевірив:
Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 5

Тема: Нормалізація бази даних.

Мета: Покращення цілісності даних шляхом структурування таблиць таким чином, щоб кожен факт зберігався лише один раз, зменшення дублювання даних та усунення аномалій оновлення/вставки/видалення.

Завдання:

1.Пошук надлишковості та аномалій: Виявлення потенційної надлишковості даних (наприклад, повторювані значення) або аномалій (проблеми вставки/оновлення/видалення) у поточній схемі.

2.Перелік функціональних залежностей: Визначення та перелік функціональних залежностей (ФЗ) для кожної проблемної таблиці.

3.Перевірка нормальних форм: Оцінка поточної нормальної форми кожної таблиці (1NF, 2NF, 3NF) на основі її функціональних залежностей та структури ключа.

4.Застосування нормалізації: Перетворення таблиць у вищі нормальні форми (до ЗНФ) для усунення часткових та транзитивних залежностей.

Аналіз початкової схеми та функціональні залежності (ФЗ):

1. Сутність "Гість" (`guest`)
Атрибути: `guest_id, name, surname, passport_details, phone, email.`

ФЗ 1.1: `guest_id → name, surname, passport_details, phone, email`
(Первинний сурогатний ключ однозначно визначає всі дані гостя).

ФЗ 1.2: `passport_details → name, surname, phone, email`
(Паспортні дані є унікальними і також можуть ідентифікувати гостя, будучи потенційним ключем).

2. Сутність "Номер та його категорія"
Якби атрибути номеру та категорії зберігались разом: `room_id, room_number, status, type_name, base_price.`

ФЗ 2.1: `room_id → room_number, status, type_name, base_price`

ФЗ 2.2: `type_name → base_price`
(Ціна залежить від категорії номера, а не від конкретного фізичного номера).

3. Сутність "Бронювання" (booking)
Атрибути: `booking_id, check_in_date, check_out_date, booking_status, guest_id, room_id.`

ФЗ 3.1: `booking_id → check_in_date, check_out_date, booking_status, guest_id, room_id`

4. Сутність "Платіж" (payment)
Атрибути: `payment_id, amount, payment_date, booking_id.`

ФЗ 4.1: `payment_id → amount, payment_date, booking_id`

<u>Перевірка нормальних форм та етапи нормалізації:</u>

Етап 1: Перша нормальна форма (1NF)
Правило: Усі атрибути містять лише атомарні (неподільні) значення, немає повторюваних груп.

Аналіз: У нашій схемі телефони (`phone`) та електронні пошти (`email`) зберігаються як єдині скалярні значення.

Висновок: Всі таблиці відповідають вимогам 1NF.

Етап 2: Друга нормальна форма (2NF)
Правило: Таблиця в 1NF, і всі неключові атрибути повністю залежать від усього первинного ключа (відсутність часткових залежностей).

Аналіз: База даних використовує сурогатні первинні ключі (`guest_id, room_id` тощо). Складених ключів немає, тому часткові залежності неможливі.

Висновок: Всі таблиці знаходяться в 2NF.

Етап 3: Третя нормальна форма (3NF)

Правило: Таблиця в 2NF, і не має транзитивних залежностей (неключовий атрибут не залежить від іншого неключового атрибута).

Аналіз та нормалізація: Як було показано у ФЗ 2.2, якби `type_name` та `base_price` знаходилися в таблиці номерів, ми мали б транзитивну залежність (`room_id` → `type_name` → `base_price`), що викликало б аномалію оновлення (зміна ціни вимагала б оновлення сотень рядків).

Рішення: Для усунення цієї залежності атрибути було розділено на дві таблиці: `room_type` (довідник категорій) та `room` (конкретні номери з посиланням `type_id`).

Висновок: Схема позбулася транзитивних залежностей і повністю відповідає 3NF.

<u>Фінальні SQL DDL-скрипти (схема у 3NF):</u>

```
-- 1. Таблиця категорій номерів
CREATE TABLE room_type (
   type_id SERIAL PRIMARY KEY,
   type_name VARCHAR(50) NOT NULL,
   description TEXT,
   base_price NUMERIC(8,2) NOT NULL CHECK (base_price > 0)
);

-- 2. Таблиця гостей
CREATE TABLE guest (
   guest_id SERIAL PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   surname VARCHAR(50) NOT NULL,
   passport_details VARCHAR(50) UNIQUE NOT NULL,
   phone VARCHAR(20),
   email VARCHAR(100) UNIQUE
);

-- 3. Таблиця номерів
CREATE TABLE room (
   room_id SERIAL PRIMARY KEY,
   room_number VARCHAR(10) UNIQUE NOT NULL,
   status VARCHAR(20) DEFAULT 'вільний',
```

```
   type_id INTEGER REFERENCES room_type(type_id) ON DELETE RESTRICT
);

-- 4. Таблиця бронювань
CREATE TABLE booking (
   booking_id SERIAL PRIMARY KEY,
   check_in_date DATE NOT NULL,
   check_out_date DATE NOT NULL,
   booking_status VARCHAR(20) DEFAULT 'підтверджено',
   guest_id INTEGER REFERENCES guest(guest_id) ON DELETE CASCADE,
   room_id INTEGER REFERENCES room(room_id) ON DELETE RESTRICT,
   CONSTRAINT chk_dates CHECK (check_out_date > check_in_date)
);

-- 5. Таблиця платежів
CREATE TABLE payment (
   payment_id SERIAL PRIMARY KEY,
   amount NUMERIC(8,2) NOT NULL CHECK (amount > 0),
   payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   booking_id INTEGER REFERENCES booking(booking_id) ON DELETE
CASCADE
);
```

<u>Результати виконання та тестування бази даних:</u>

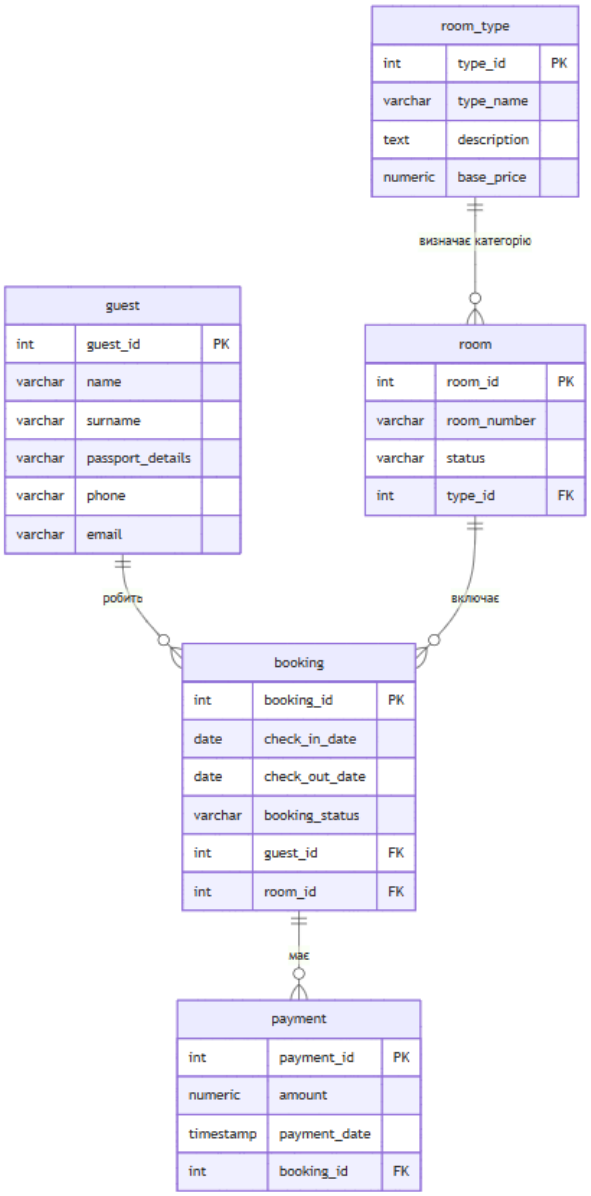
Після застосування DDL-скриптів та наповнення таблиць тестовими даними, було виконано запит з використанням `JOIN` для перевірки цілісності зв'язків між нормалізованими таблицями.

Data Output Messages Notifications

Showing rows: 1 to 2

	№ Бронювання integer	Ціть text	Кімната character varying (10)	Категорія character varying (50)	Дата заїзду date	Дата виїзду date
1	1	Олександр Іван...	101	Стандарт	2026-05-01	2026-05-05
2	2	Марія Петренко	205	Люкс	2026-05-10	2026-05-12

ER-Діаграма нормалізованої бази даних:



Висновок: час виконання лабораторної роботи було проаналізовано схему бази даних «Готель». Було визначено всі функціональні залежності атрибутів. Завдяки правильному проєктуванню з використанням сурогатних ключів та виокремленню довідкових сутностей (таких як `room_type`), база даних була приведена до Третьої нормальної форми (3NF). Усунення транзитивних залежностей гарантує відсутність аномалій додавання, оновлення чи видалення, а також запобігає дублюванню даних. Працездатність нормалізованої схеми успішно перевірена в СУБД PostgreSQL.